

JOBSHEET PRAKTIKUM

Mata Kuliah: Pemrograman Framework

Topik: Implementasi Login Google Provider dengan NextAuth.js + Firebase

Tujuan Pembelajaran

Setelah menyelesaikan praktikum ini, mahasiswa mampu:

1. Mengkonfigurasi Google OAuth melalui Google Cloud Console
 2. Mengintegrasikan Google Provider ke NextAuth.js
 3. Mengelola session dan JWT callback
 4. Menyimpan data user Google ke Firebase Firestore
 5. Mengimplementasikan multi-role (member & admin)
 6. Menampilkan avatar dan data profil user
 7. Memahami alur autentikasi OAuth berbasis provider
-

PRAKTIKUM: Google Login dengan NextAuth.js

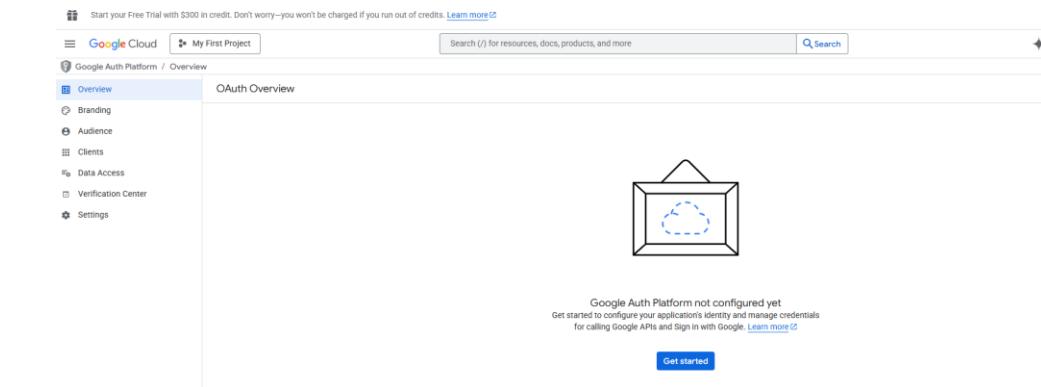
A. Persiapan

1 Pastikan Project Sudah Memiliki:

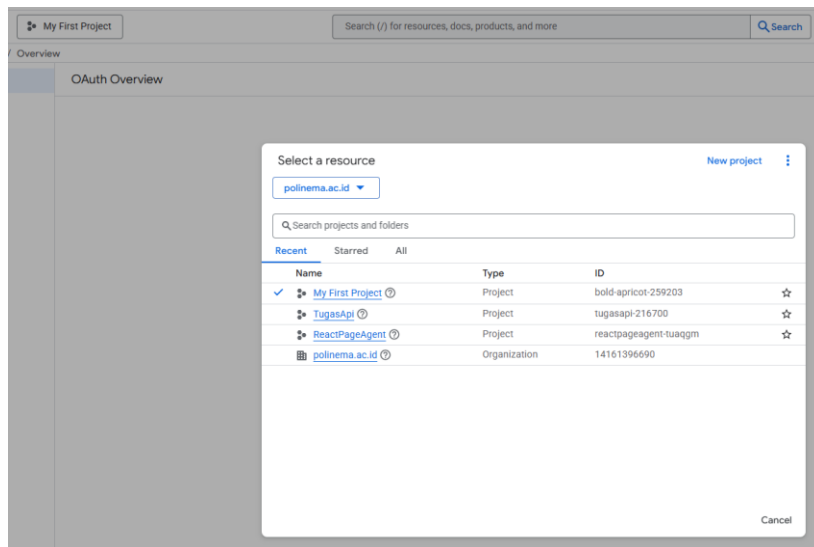
- Next.js (App Router atau Pages Router)
 - NextAuth.js terinstall
 - Firebase Firestore terkoneksi
 - Sistem login credential sudah berjalan (opsional)
-

B. Konfigurasi Google OAuth

- Langkah 1 – Masuk ke Google Cloud Console Buka:
 - <https://console.cloud.google.com/apis/credentials>



- Langkah 2 – Buat Project Baru
- Klik New Project



- Nama project: MyAppNext

New Project

You have 17 projects remaining in your quota. Request an increase or delete projects. [Learn more](#)

[Manage Quotas](#)

Project name *
MyAppNext

Project ID: myappnext-488013. It cannot be changed later. [Edit](#)

Organization *
polinema.ac.id

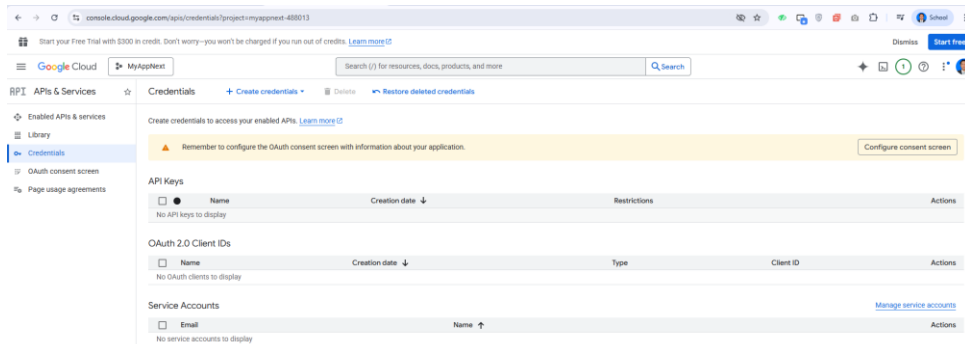
Select an organization to attach it to a project. This selection can't be changed later.

Parent resource *
polinema.ac.id [Browse](#)

Select the organization or folder where this resource will be created

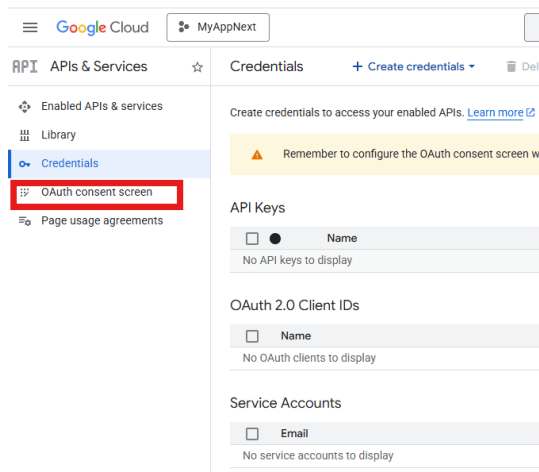
[Create](#) [Cancel](#)

- Klik Create
- Setelah berhasil klik <https://console.cloud.google.com/apis/credentials> pastikan projectnya MyAppNext

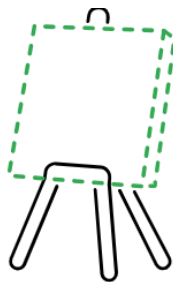


Langkah 3 – Konfigurasi OAuth Consent Screen

1. Pilih OAuth consent screen



2. Pilih Get Started



Google Auth Platform not configured yet
Get started to configure your application's identity and manage credentials
for calling Google APIs and Sign in with Google. [Learn more](#)

Get started

3. Maka akan muncul seperti berikut dan isikan

1 App Information

App name *
my-app

The name of the app asking for consent

User support email *
[Redacted]

For users to contact you with questions about their consent. [Learn more](#)

[Next](#)

2 Audience

3 Contact Information

4 Finish

[Create](#) [Cancel](#)

✓ App Information

2 Audience

☐ Internal ⓘ

Only available to users within your organization. You will not need to submit your app for verification. [Learn more about user type](#)

☒ External ⓘ

Available to any test user with a Google Account. Your app will start in testing mode and will only be available to users you add to the list of test users. Once your app is ready to push to production, you may need to verify your app. [Learn more about user type](#)

[Next](#)

3 Contact Information

4 Finish

[Create](#) [Cancel](#)

Project configuration

✓ App Information

✓ Audience

3 Contact Information

Email addresses *
[Redacted]

These email addresses are for Google to notify you about any changes to your project.

[Next](#)

4 Finish

[Create](#) [Cancel](#)

Project configuration

- ✓ App Information
- ✓ Audience
- ✓ Contact Information
- 4 Finish

☒ I agree to the [Google API Services: User Data Policy](#)

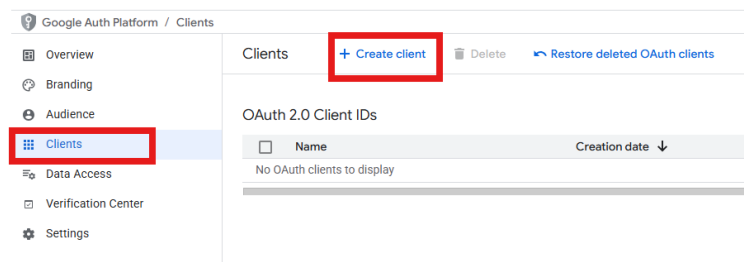
[Continue](#)

[Create](#) [Cancel](#)

- Klik create

Langkah 4 – Buat OAuth Credentials

1. Klik create client pada Clients



← Create OAuth client ID

A client ID is used to identify a single app to Google's OAuth servers. If your app runs on multiple platforms, each will need its own client ID. See [Setting up OAuth 2.0](#) for more information. [Learn more](#) about OAuth client types.

Application type *
Web application

Name *
Web client 1

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.

i The domains of the URIs you add below will be automatically added to your [OAuth consent screen](#) as [authorized domains](#).

Authorized JavaScript origins ⓘ
For use with requests from a browser

URIs 1 *
http://localhost:3000

[+ Add URI](#)

Authorized redirect URIs ⓘ
For use with requests from a web server

URIs 1 *
http://localhost:3000/api/auth/callback/google

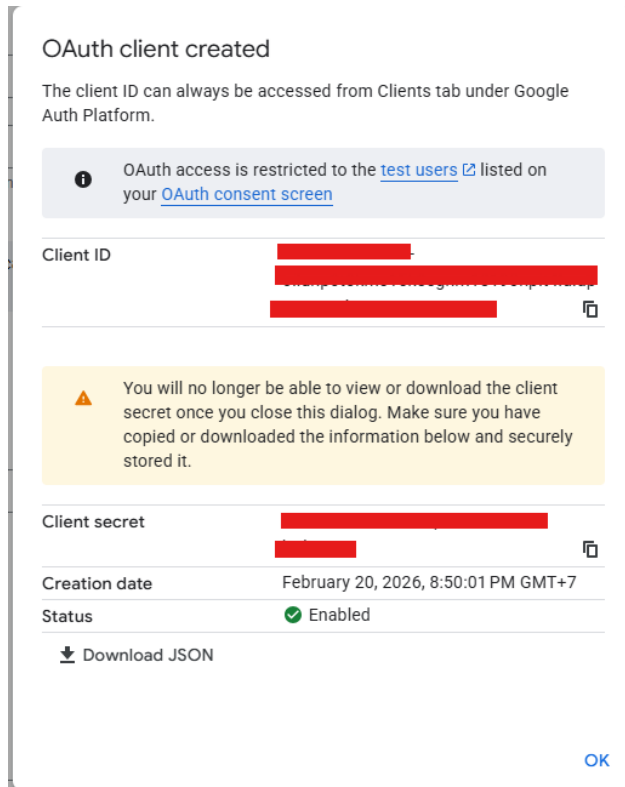
[+ Add URI](#)

Note: It may take 5 minutes to a few hours for settings to take effect

[Create](#) [Cancel](#)

✦ C. Tambahkan Environment Variables

1. Copy dan paste client ID dan Client secret ke .env



```
.env.local x
16.praktikum-LoginGoogle > my-app > .env.local
1 FIREBASE_API_KEY=
2 FIREBASE_AUTH_DOMAIN=
3 FIREBASE_PROJECT_ID=
4 FIREBASE_STORAGE_BUCKET=
5 FIREBASE_MESSAGING_SENDER_ID=
6 FIREBASE_APP_ID=
7
8 REVALIDATE_TOKEN=12345678
9
10 // gunakan website https://www.convertsimple.com/random-base64-generator/ untuk membuat secret key yang panjang dan acak,
11 // lalu masukkan ke dalam variabel NEXTAUTH_SECRET
12 NEXTAUTH_SECRET=
13
14
15 GOOGLE_CLIENT_ID=
16 GOOGLE_CLIENT_SECRET=
```

✦ D. Konfigurasi Google Provider di NextAuth dan Handle Callback JWT & Session

1. Buka file [...nextauth].ts pada folder api/auth dan modifikasi menjadi berikut

```
16.praktikum-LoginGoogle > my-app > src > pages > api > auth > TS [...nextauth].ts > authOptions > callbacks > session
1  import { signIn } from "@utils/db/servicefirebase";
2  import NextAuth, { NextAuthOptions } from "next-auth";
3  import CredentialsProvider from "next-auth/providers/credentials";
4  import GoogleProvider from "next-auth/providers/google";
5
6
7  export const authOptions: NextAuthOptions = {
8    session: {
9      strategy: "jwt",
10    },
11    secret: process.env.NEXTAUTH_SECRET,
12    providers: [
13      CredentialsProvider({
14      }),
15      GoogleProvider({
16        clientId: process.env.GOOGLE_CLIENT_ID || "",
17        clientSecret: process.env.GOOGLE_CLIENT_SECRET || "",
18      }),
19    ],
20    callbacks: {
21      async jwt({ token, account, profile, user }: any) {
22        if (account?.provider === "credentials" && user) {
23          token.email = user.email;
24          token.fullname = user.fullname;
25          token.role = user.role;
26        }
27        // Jika login dengan Google, tambahkan informasi yang diperlukan ke token
28        if (account?.provider === "google") {
29          const data = {
30            fullname: user.name,
31            email: user.email,
32            image: user.image,
33            type: account.provider,
34          };
35          // console.log("Google login data", { data });
36          token.fullname = data.fullname;
37          token.email = data.email;
38          token.image = data.image;
39          token.type = data.type;
40        }
41        return token;
42      },
43    },
44  };
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
```

```

16.praktikum-LoginGoogle > my-app > src > pages > api > auth > TS [...nextauth].ts > authOptions > callbacks > session
7   export const authOptions: NextAuthOptions = {
49     callbacks: {
50       async jwt({ token, account, profile, user }: any) {
68         token.type = data.type;
69       },
70     },
71     return token;
72   },
73   Windsurf: Refactor | Explain | Generate JSDoc | X
74   async session({ session, token }: any) {
75     if (token.email) {
76       session.user.email = token.email;
77     }
78     if (token.fullname) {
79       session.user.fullname = token.fullname;
80     }
81     if (token.image) {
82       session.user.image = token.image;
83     }
84     if (token.role) {
85       session.user.role = token.role;
86     }
87     if (token.type) {
88       session.user.type = token.type;
89     }
90     // console.log('session callback', { session, token })
91     return session;
92   },
93   pages: {
94     signIn: "/auth/login",
95   },
96 };
97 export default NextAuth(authOptions);
98

```

E. Tambahkan Button Login Google

1. Modifikasi file index.tsx pada folder views/auth/login

```

16.praktikum-LoginGoogle > my-app > src > views > auth > login > index.tsx > ...
7   const Tampilanlogin = () => {
71     </div>
72   </div>
73   <button
74     type="submit"
75     className={style.login_form_item_button}
76     disabled={isLoading}
77   >
78     {isLoading ? "Loading..." : "login"}
79   </button>{""}
80   <br /> <br />
81   <button
82     onClick={() => signIn("google", { callbackUrl, redirect: false })}
83     className={style.login_form_item_button}
84     disabled={isLoading}
85   >
86     {isLoading ? "Loading..." : "sign in with google"}
87   </button>
88 </form>
89 <br />
90 <p className={style.login_form_item_text}>
91   tidak punya {"'"} akun?{"'"}
92   <Link href="/auth/register">Ke Halaman Register</Link>
93 </p>
94 </div>
95 </div>
96 </>
97 );
98 };
99
100 export default Tampilanlogin;

```

- Jalankan browser localhost:3000/auth/login masuk melalui sign in with google. Jika berhasil maka akan terhubung dengan akun google.

Halaman login

Email

Password

login

sign in with google

tidak punya ' akun? Ke Halaman Register

Note: data akun google tidak tersimpan dalam database

3. Menampilkan image dari google

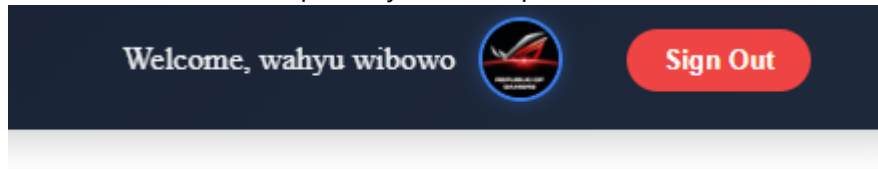
- o Buka file index.tsx dan tambahkan code berikut

```
16.praktikum-LoginGoogle > my-app > src > components > layouts > navbar > index.tsx > Navbar
1 { import styles from './navbar.module.scss';
2 import { signIn, signOut, useSession } from "next-auth/react"
3
4 const Navbar = () => {
5   const {data}:any = useSession()
6   //const { data: session } = useSession()
7   // console.log("session", session)
8   return (
9     <div className={styles.navbar}>
10       <div className={styles.navbar__brand}>
11         MyApp
12       </div>
13
14       <div className={styles.navbar__right}>
15         {data ? (
16           <>
17             <div className={styles.navbar__user}>
18               Welcome, {data.user?.fullname}
19               {data.user.image && (
20                 <img
21                   src={data.user.image}
22                   alt={data.user.fullname}
23                   className={styles.navbar__user__image}
24                 />
25               )}
26             </div>
27             <button
28               className={`${styles.navbar__button} ${styles["navbar__button--danger"]}`}
29               onClick={() => signOut()}
30             >
31               Sign Out
32             </button>
33           </>
34         ) : (
35           <button
36             className={`${styles.navbar__button} ${styles["navbar__button--primary"]}`}
37             onClick={() => signIn()}
38           >
39             Sign In
40           </button>
41         )}
42       </div>
43     </div>
44   );
45 };
46
47 export default Navbar;
```

- Buka file navbar.module.css dan tambahkan code berikut

```
16.praktikum-LoginGoogle > my-app > src > components > layouts > navbar > # navbar.module.css > { } @media (max-width: 768px)
70
71 /* Responsive */
72 > @media (max-width: 768px) { ...
80
81 .navbar_user {
82   display: flex;
83   align-items: center;
84   gap: 12px;
85   font-size: 16px;
86   font-weight: 500;
87   opacity: 0.95;
88 }
89
90 /* Avatar image */
91 .navbar__user__image {
92   width: 42px;
93   height: 42px;
94   border-radius: 50%;
95   object-fit: cover;
96   border: 2px solid #3b82f6;
97   box-shadow: 0 0 10px rgba(59, 130, 246, 0.4);
98   transition: all 0.3s ease;
99 }
100
101 .navbar__user__image:hover {
102   transform: scale(1.1);
103   box-shadow: 0 0 15px rgba(59, 130, 246, 0.8);
104 }
```

- Jika berhasil maka tampilannya akan seperti berikut



✿ G. Simpan Data Google ke Database

- Buka file servicefirebase.ts pada folder src/utills/db/ dan tambahkan beberapa kode berikut

```
16.praktikum-LoginGoogle > my-app > src > utills > db > TS servicefirebase.ts > signInWithGoogle
1  import {
2    getFirestore,
3    collection,
4    getDocs,
5    Firestore,
6    getDoc,
7    doc,
8    query,
9    addDoc,
10   where,
11   updateDoc,
12 } from "firebase/firestore";
13 import app from "../firebase";
14 import bcrypt from "bcrypt";
15
```

- Tambahkan juga code berikut

16.praktikum-LoginGoogle > my-app > src > utils > db > TS servicefirebase.ts > signInWithGoogle

```
Windsurf: Refactor | Explain | Generate JSDoc | X
47 > export async function signUp(...)
92 }
93
Windsurf: Refactor | Explain | Generate JSDoc | X
94 export async function signInWithGoogle(userData: any, callback: any) {
95   try {
96     const q = query(
97       collection(db, "users"),
98       where("email", "==", userData.email),
99     );
100
101     const querySnapshot = await getDocs(q);
102     const data: any = querySnapshot.docs.map((doc) => ({
103       id: doc.id,
104       ...doc.data(),
105     }));
106
107     if (data.length > 0) {
108       // User sudah ada, update data
109       userData.role = data[0].role;
110       await updateDoc(doc(db, "users", data[0].id), userData);
111       callback({
112         status: true,
113         message: "User registered and logged in with Google",
114         data: userData,
115       });
116     } else {
117       // User baru, tambah data
118       userData.role = "member";
119       await addDoc(collection(db, "users"), userData);
120       callback({
121         status: true,
122         message: "User registered and logged in with Google",
123         data: userData,
124       });
125     }
126   } catch (error: any) {
127     // Tangani error di sini
128     callback({
129       status: false,
130       message: "Failed to register user with Google",
131     });
132   }
}
```

- Panggil Service di JWT Callback buka file [...nextAuth].ts

16.praktikum-LoginGoogle > my-app > src > pages > api > auth > TS [...nextauth].ts > ...

```
7 export const authOptions: NextAuthOptions = {
47 },
48
49 callbacks: {
Windsurf: Refactor | Explain | Generate JSDoc | X
50   async jwt({ token, account, profile, user }: any) {
51     if (account?.provider === "credentials" && user) {
52       token.email = user.email;
53       token.fullname = user.fullname;
54       token.role = user.role;
55     }
56
57     // Jika login dengan Google, tambahkan informasi yang diperlukan ke token
58     if (account?.provider === "google") {
59       const data = {
60         fullname: user.name,
61         email: user.email,
62         image: user.image,
63         type: account.provider,
64       };
65
66       await signInWithGoogle(data, (result: any) => {
67         // Pastikan mengecek result.status sesuai dengan object yang dikirim
68         if (result.status) {
69           token.fullname = result.data.fullname;
70           token.email = result.data.email;
71           token.image = result.data.image;
72           token.type = result.data.type;
73           token.role = result.data.role;
74         }
75       });
76     }
77     return token;
78   },
Windsurf: Refactor | Explain | Generate JSDoc | X
```

- Jalankan browser dan login menggunakan akun google setelah cek di firebase, jika data akun googlenya masuk ke database maka anda telah berhasil

✦ h. Pengujian

Skenario	Hasil yang Diharapkan
Login Google pertama kali	Data tersimpan di Firestore
Login Google kedua kali	Data diupdate
User role member akses /admin	Redirect
User role admin akses /admin	Bisa masuk
Avatar tampil	Ya

✦ Analisis & Diskusi

1. Apa perbedaan login credential dan login Google?
2. Mengapa data Google tetap perlu disimpan ke database?
3. Apa fungsi JWT callback?
4. Mengapa perlu multi-role?
5. Apa risiko jika tidak menyimpan user ke database?

✦ Tugas Mandiri

1. Tambahkan role editor
2. Buat halaman khusus editor
3. Tambahkan provider GitHub
4. Refactor service agar reusable
5. Gunakan next/image untuk optimasi avatar

✦ Kesimpulan

Pada praktikum ini mahasiswa telah:

- ✅ Mengimplementasikan Google OAuth
- ✅ Mengintegrasikan dengan NextAuth.js

- ✓ Menyimpan user ke Firestore
- ✓ Mengelola JWT dan Session
- ✓ Mengimplementasikan Multi-Role
- ✓ Menampilkan avatar profil